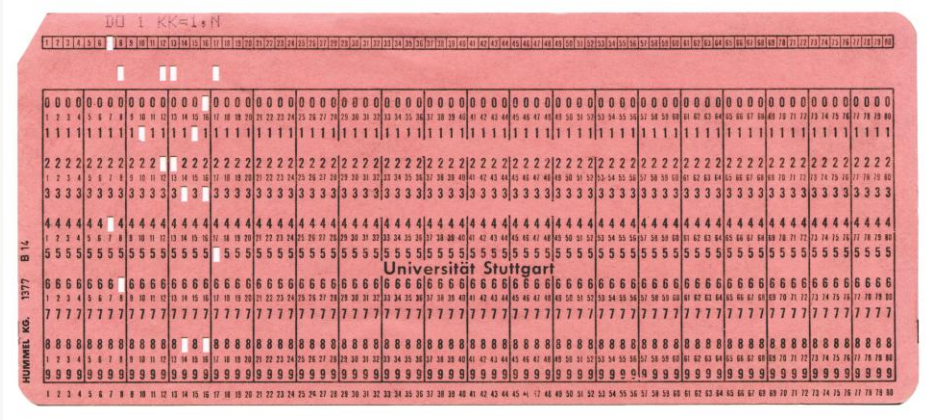
Karakterkódolás és algoritmusok

Karakterek kódolása

ASCII, EBCDIC, Unicode
Utasítások és algoritmusleírás



Programbevitel lyukkártyán

A szám és a számjegy karaktere

5 **'5'**

Ugyanaz a jel a képernyőn két különböző értelmezést takarhat.

A bitminta jelentését az értelmezés adja.

Karakterek, utasítások, algoritmusok

55 perc

Karakterkódolás
ASCII és EBCDIC
Unicode és UTF formátumok

5 + 25 perc

Rövid szünet
Gépi utasítások
CISC és RISC

35 perc

Algoritmus és program
Leíró eszközök
Közös példák

A karaktertől a megjelenő jelig

Szint	Példa az A betűre
Karakter	Latin nagy A
Kódpont	Unicode: U+0041
Kódolás és bájtok	UTF-8: 41 hexadecimálisan
Megjelenítés	A betűkészlet kiválasztja a rajzolatot

A betű képe és a tárolt kód külön szerepet tölt be.

Hét biten 128 kód fér el

$$2^7 = 128$$

ASCII: 0–127
Bájtban tárolva a legfelső bit 0.

ASCII kódok: 00–7F hexadecimálisan.

Nyomtatható jelek és vezérlőkódok

0–31

Vezérlőkódok
Például TAB, LF, CR

32–126

Szóköz, betűk,
számjegyek
és írásjelek

127

DEL
Történeti törlésjel

Egy kód akkor is hordozhat jelentést, ha nem betűként látszik.

Az A kódjának kiolvasása

Karakter	Decimális	Hexadecimális	Bináris bájt
A	65	41	01000001
B	66	42	01000010
a	97	61	01100001
0	48	30	00110000
SP	32	20	00100000

Ugyanazt a kódértéket több számrendszerben írjuk le.

Az ABC szöveg bájtjai

A

65_{10}
41_{16}
01000001_{2}

B

66_{10}
42_{16}
01000010_{2}

C

67_{10}
43_{16}
01000011_{2}

ABC ASCII és UTF-8 kódolásban: 41 42 43.

A számjegy kódja és a számérték

Egész szám

int x = 5;

A számításban
ötöt jelent.

Karakter

char c = '5';

ASCII / Unicode kódja 53.
A számjegyértéke 5.

'5' − '0' = 53 − 48 = 5.

A sorváltásnak is van kódja

Jel	Decimális	Jelentés
HT / TAB	9	Vízszintes tabulátor
LF	10	Soremelés
CR	13	Kocsi vissza
SP	32	Szóköz

A vezérlőkódot a feldolgozó program értelmezi.

A magyar szöveg több karaktert igényel

árvíztűrő tükörfúrógép

Az alap ASCII készletből hiányoznak a magyar ékezetes betűk.

Szövegcseréhez a karakterkészletet és a kódolást is ismerni kell.

A nyolcadik bit többféle bővítést hozott

256 hely

Egy nyolcbites kódlap legfeljebb 256 pozíciót ad.

Több kiosztás

ISO-8859-2, Windows-1250
és más kódlapok

Eltérő jelentés

Ugyanaz a bájt más karaktert jelenthet.

A kódlap azonosítója nélkül a bájtsorozat félreérthető.

Egy másik karakterkódolási család

Extended Binary Coded Decimal Interchange Code

Nyolcbites kódlapok.
IBM nagyszámítógépes környezet.
Példánk: IBM-037.



Azonos karakterhez más kódérték tartozhat.

ASCII és EBCDIC IBM-037

Karakter	ASCII hex	IBM-037 hex
SP	20	40
A	41	C1
B	42	C2
a	61	81
0	30	F0
9	39	F9

Az átkódolás jelentést őrző megfeleltetés.

A betűk kódjai több csoportot alkotnak

Karaktercsoport	Hexadecimális tartomány
A–I	C1–C9
J–R	D1–D9
S–Z	E2–E9
0–9	F0–F9

A kódsorrend és a nyelvi ábécérend külön fogalom.

Két bájtsorozat megfejtése

ASCII

41 42 43

Melyik három karakter?

EBCDIC IBM-037

C1 C2 C3

Melyik három karakter?

Először a kódolás nevét, utána a kódértéket értelmezzük.

Azonos szöveg eltérő bájtokkal

Kódolás	Tárolt bájtok hexadecimálisan	Szöveg
ASCII	41 42 43	ABC
UTF-8	41 42 43	ABC
EBCDIC IBM-037	C1 C2 C3	ABC

A bájtok önmagukban nem nevezik meg a kódolásukat.

Közös kódtér a különböző írásrendszerekhez

Latin, görög, arab, cirill
és sok más írásrendszer.

Közös karakterazonosítók.
Szabályok a feldolgozáshoz.



A Unicode és az UTF-8 fogalma összetartozik, de eltérő szintet jelöl.

Az U+ jelölés hexadecimális azonosítót ad

Karakter	Kódpont	Decimális érték
A	U+0041	65
ő	U+0151	337
€	U+20AC	8364
Mosolygó arc	U+1F600	128512

A kódpontot a választott UTF alak kódolja tovább.

Kódpont, kódegység és látható szövegelem

Kódpont

Hely a Unicode kódtérben.
U+0000–U+10FFFF.

Kódegység

A kódolás alapegysége.
UTF-8: 8 bit.
UTF-16: 16 bit.

Látható szövegelem

Egy vagy több kódpont is együtt alkothatja.

Egy látható jel darabszáma nem minden esetben string.Length.

UTF-8, UTF-16 és UTF-32

Alak	Kódegység	Egy skalárérték mérete
UTF-8	8 bit	1–4 bájt
UTF-16	16 bit	1 vagy 2 egység, azaz 2 vagy 4 bájt
UTF-32	32 bit	1 egység, azaz 4 bájt

Az UTF számjegye a kódegység bitméretét jelzi.

A kezdő bitek jelzik a bájtok szerepét

Bájtszám	Bájtok mintája
1	0xxxxxxx
2	110xxxxx 10xxxxxx
3	1110xxxx 10xxxxxx 10xxxxxx
4	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

A 10 kezdetű bájt folytatásbájt.

Az ő betű két bájtot kap

Lépés	Érték
Kódpont	U+0151 = 337
Adatbitek 5 + 6 csoportban	00101 010001
UTF-8 keretbe illesztve	11000101 10010001
Bájtok hexadecimálisan	C5 91

Egy kódpont reprezentációja több bájtból is állhat.

Az ár szó két kódpont és három bájt

Karakter	Unicode kódpont	UTF-8 bájtok hex
á	U+00E1	C3 A1
r	U+0072	72
A teljes szó	2 kódpont	C3 A1 72

A szöveghossz mérésénél mindig nevezzük meg az egységet.

A string.Length UTF-16 egységeket számol

```
string s = "A\U0001F600";

Console.WriteLine(s.Length);
// 3

Console.WriteLine(
    Encoding.UTF8.GetByteCount(s));
// 5
```

A + U+1F600

2 kódpont
3 UTF-16 kódegység
5 UTF-8 bájt

A kódpont és a C# char külön fogalom.

Az é többféle kódpontsorozattal is leírható

Előre összeállított

é

U+00E9
1 UTF-16 egység

Összetett alak

e + kombináló ékezet

U+0065 U+0301
2 UTF-16 egység

A megjelenés azonossága nem garantál azonos bájtsorozatot.

A rossz kódolásnév elrontja a szöveget

ár → Ã¡r

A C3 A1 72 UTF-8 bájtokat hibásan Windows-1252 szerint olvastuk.

A kódoló és a dekódoló ugyanazt a megállapodást kövesse.

Karakterkódolási gyorspróba

1. Az Aő UTF-8 szöveg hány bájt?
2. Az IBM-037 és az ASCII ugyanazokat a kódokat használja?
3. A Unicode és az UTF-8 ugyanazt jelenti?

A következő blokkban az utasítások jelentését vizsgáljuk.

Öt perc szünet

5 perc

Utána: gépi utasítások, CISC és RISC.

A gépi utasítás műveletet ír elő

Művelet

Mit hajtson végre
a processzor?

Operandusok

Mely adatokkal
vagy címekkel dolgozzon?

Állapotváltozás

Regiszter, memória
vagy vezérlés változhat.

A bitmintát az utasításkészlet szabályai értelmezik.

Műveleti kód és operandusok

Szemléltető utasítás

ADD R0, R1, R2

R0 új értéke:
R1 + R2

A mezők szerepe

ADD: összeadás
R0: célregiszter
R1 és R2: források

A szöveges assemblerjelölés a gépi kód olvasható megfelelője.

Adatmozgatás, számítás és vezérlés

Csoport	Szemléltető művelet	Feladat
Adatmozgatás	LOAD / STORE	Memória és regiszter között
Aritmetika	ADD / SUB	Összeadás vagy kivonás
Logika	AND / OR	Bitenkénti művelet
Vezérlés	Feltételes ugrás	A következő utasítás kiválasztása

A magas szintű program egy sora több gépi utasítást igényelhet.

Az utasítás feldolgozásának alapciklusa

Lekérés

A processzor az utasítás címe alapján kódot olvas.

Értelmezés

Felismeri a műveletet és az operandusokat.

Végrehajtás

Elvégzi a műveletet, frissíti az állapotot.

A feldolgozás ismétlődik a program vezérlése szerint.

Az ISA és a processzor megvalósítása

ISA

A program számára látható utasítások és szabályok.

Példa: x86-64 vagy RV32I.

Megvalósítás

Hogyan építi fel és hajtja végre ezeket a processzor.

Pipeline, gyorsítótár, belső lépések.

Az utasításkészlet neve önmagában nem mondja meg a futásidőt.

Összetettebb utasításkészleti hagyomány

A rövidítés

Complex Instruction Set Computer

Jellemző irány

Összetettebb műveletek és címzési lehetőségek.

Példa

Az x86 család. Változó hosszúságú utasításkódok.

A CISC címke az utasításkészlet tervezési hagyományát írja le.

Egyszerűbb, szabályosabb utasítások hagyománya

A rövidítés

Reduced Instruction Set Computer

Jellemző irány

Regisztereken végzett aritmetika.
Külön memóriaátvitel.

Példa

RISC-V RV32I.
Load/store felépítés.

A memóriaelérést és a számítást többnyire külön utasítás végzi.

Azonos művelet két szemléltető leírása

```
// CISC jellegű modell
ADD [x], R1

// RISC jellegű modell
LOAD R2, [x]
ADD R2, R2, R1
STORE [x], R2
```

Feladat: x értékéhez hozzáadni R1 értékét.

Mindkét modell ugyanazt a kívánt állapotot írja le.

Az utasításszám és a futási idő eltérő mérőszám.

A CISC és RISC jellemző irányai

Szempont	CISC hagyomány	RISC hagyomány
Műveletek	Összetettebb alakok is	Egyszerűbb, szabályos alakok
Memória	Számításban is lehet operandus	Tipikusan külön load/store
Kódhossz	Gyakran változó	Alapforma gyakran rögzített
Példa	x86	RISC-V

A konkrét architektúrát és a konkrét programot kell vizsgálni.

Három félrevezető általánosítás

Állítás	Miért nem általános szabály?
Minden RISC utasítás egyciklusos	A végrehajtás a művelettől és a hardvertől függ.
Kevesebb utasítás mindig gyorsabb	A műveletek és memóriaelérések költsége eltér.
A RISC kód mindig azonos hosszú	Léteznek tömörített és változó hosszú kódolások.

A CISC–RISC különbségből önmagában nem következik teljesítménysorrend.

Utasítás és végrehajtás

1. Melyik rész nevezi meg a műveletet az ADD R0, R1, R2 példában?
2. Miért nem azonos egy C# forrássor és egy gépi utasítás?
3. Mit jelent a load/store felépítés?

A megoldás menete és annak megvalósítása

Algoritmus

Egy feladat megoldásának pontosan megadott lépései.

Program

Az algoritmus konkrét programnyelvi megvalósítása és a szükséges keret.

A programnyelv szintaxisa és az algoritmus logikája külön vizsgálható.

Pontos lépések, meghatározott eredmény

Egyértelműség

A lépések jelentése pontosan rögzített.

Végrehajthatóság

A lépések elvégezhető műveletekből állnak.

Befejezés

A tárgyalt véges feladat megengedett bemenetein a folyamat véget ér.

A helyességet a megengedett bemenetekre és a célra értelmezzük.

Két egész szám nagyobbikának kiválasztása

Kérdés	Rögzített válasz
Bemenet	Két egész szám: a és b
Kimenet	A nagyobbik érték
Egyenlőség	A közös érték
Próbák	7 és 3, 4 és 4, −2 és −5

A leírások mind ugyanennek a specifikációnak felelnek meg.

A nagyobbik érték lépései

1. Olvassa be a és b értékét.
2. Hasonlítsa össze őket.
3. Ha a ≥ b, az eredmény a. Egyébként b.
4. Írja ki az eredményt.

Az egyenlő bemenetek esete is része a megoldásnak.

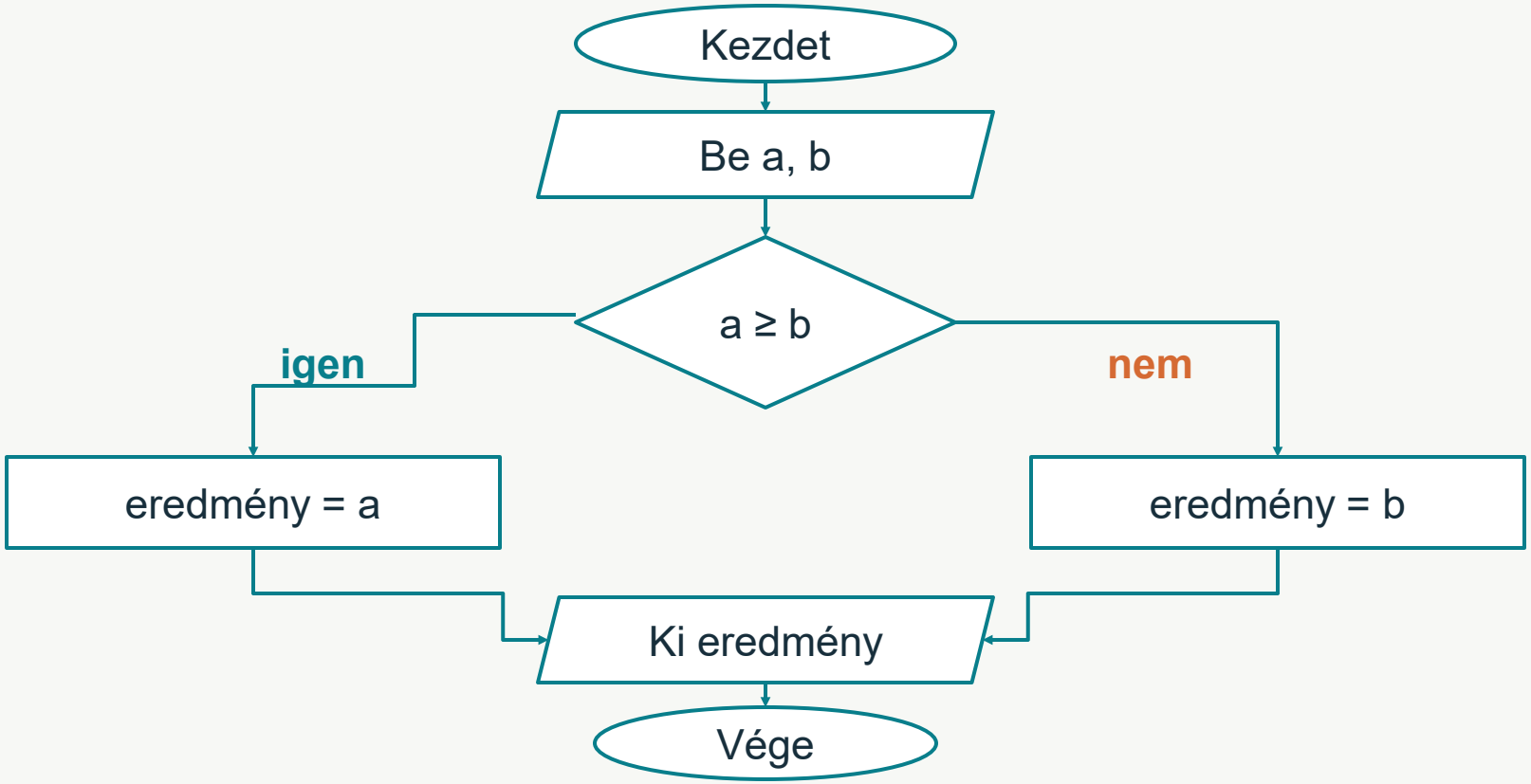
A nagyobbik érték pszeudokódja

```
BE a, b
HA a >= b AKKOR
    eredmeny <- a
KÜLÖNBEN
    eredmeny <- b
ELÁGAZÁS VÉGE
KI eredmeny
```

A logikai szerkezet látszik.

A jelölés nyelvfüggetlen.
Az értékadás jele itt: <-

A döntés két ága és közös folytatása



Rombusz: feltétel Téglalap: értékadás Paralelogramma: be és ki

Az elágazás egymásba ágyazott blokkokkal

Be a és b	
a ≥ b	
IGEN	NEM
eredmény = a	eredmény = b
Ki eredmény	

A blokkok elhelyezése adja meg a szerkezetet.

Ugyanez egy rövid függvényben

```
static int Nagyobbik(int a, int b)
{
    if (a >= b) return a;
    return b;
}

Console.WriteLine(Nagyobbik(7, 3));
// 7
```

A return a teljes függvény végrehajtását befejezi.

A második return csak a < b esetén fut le.

A logika ugyanaz, mint a pszeudokódban.

Szekvencia, elágazás és ismétlés

Szekvencia

A lépések egymás után következnek.

Elágazás

A feltétel alapján másik ág fut le.

Ismétlés

Egy lépéssor feltétel szerint ismétlődik.

Az 1-től n-ig terjedő számok összege

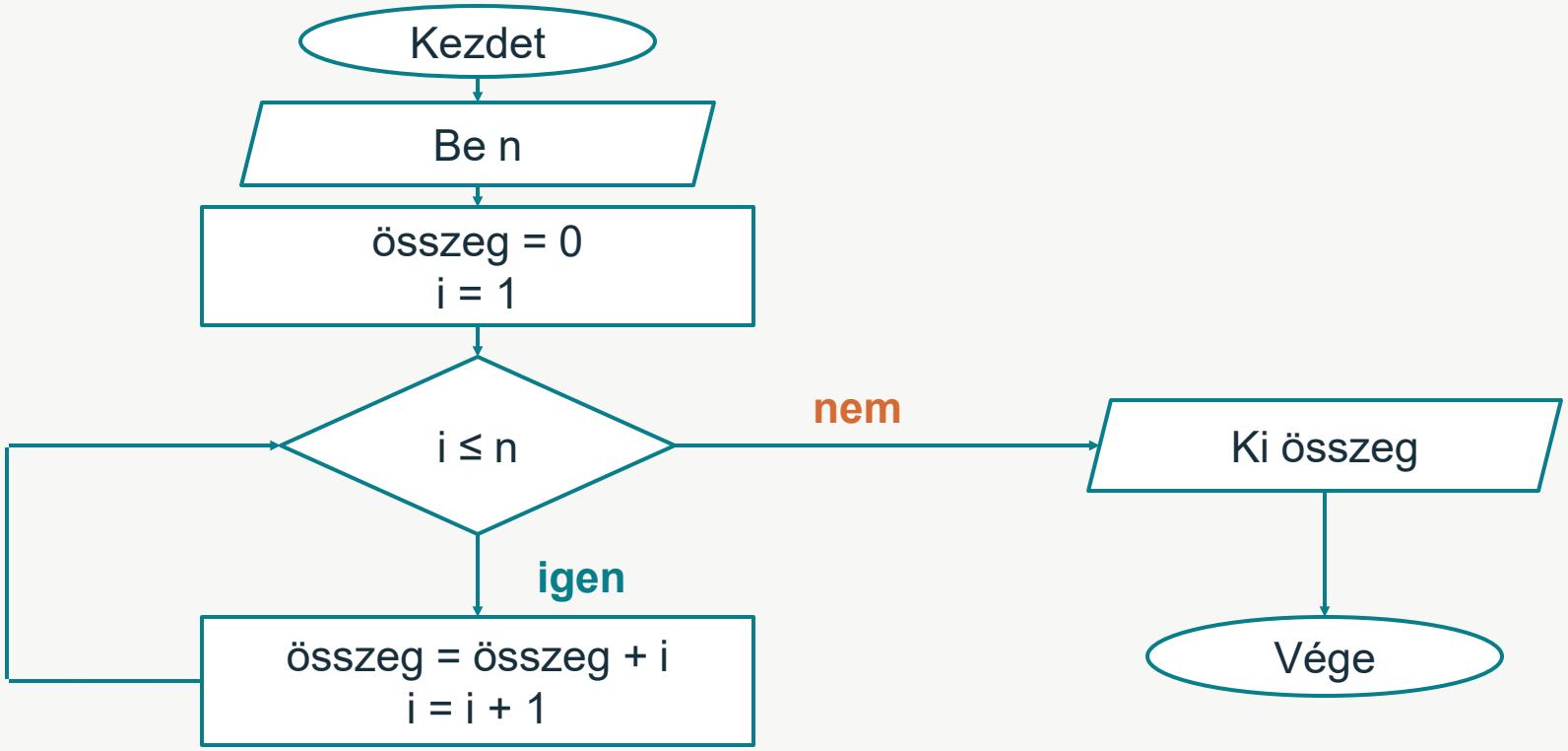
```
BE n
osszeg <- 0
i <- 1
AMÍG i <= n
    osszeg <- osszeg + i
    i <- i + 1
CIKLUS VÉGE
KI osszeg
```

Bemenet: n ≥ 0 egész.
Kimenet: 1 + 2 + … + n.

n = 0 esetén az összeg 0.
A ciklus egyszer sem fut.

Minden körben változik a ciklusvezérlő érték.

A feltételhez visszatérő végrehajtás



Előfeltétel: n ≥ 0 egész

Az összegzés állapotai n = 4 esetén

Állapot	i	összeg
Kezdet	1	0
1 hozzáadása után	2	1
2 hozzáadása után	3	3
3 hozzáadása után	4	6
4 hozzáadása után	5	10

Négy lefutás után az eredmény 10.

Ciklushatár és elmaradó léptetés

Hibás határ

AMÍG i < n

n = 4 esetén:
1 + 2 + 3 = 6

Hiányzó növelés

i nem változik.

Pozitív n esetén a feltétel igaz maradhat.

A feltételt és a változó frissítését együtt ellenőrizzük.

Ugyanaz a lépéssor más jelöléssel

```
long osszeg = 0;
for (int i = 1; i <= n; i++)
{
    osszeg += i;
}
Console.WriteLine(osszeg);
```

Előfeltétel a kódhoz:
$0 \le n \le 1\,000\,000$.

A long összeg ebben a tartományban elfér.

A helyes programhoz a típusok és a bemeneti tartomány is hozzátartozik.

A leírási formák eltérő szempontokat mutatnak

Eszköz	Mit tesz könnyen követhetővé?
Természetes nyelv	A feladat jelentését és az alaplépéseket
Pszeudokód	A logikát és a vezérlési szerkezetet
Folyamatábra	Az elágazásokat és a bejárási útvonalat
Struktogram	A strukturált blokkok egymásba ágyazását
Programkód	A konkrét nyelvi megvalósítást

Három szám legnagyobb értéke

1. Bemenet: három egész szám, a, b és c.
2. Kimenet: a legnagyobb érték.
3. Készüljön pszeudokód. Próbák: (7, 3, 5), (4, 4, 4), (−2, −5, −3).

Az aktuális legnagyobb érték frissítése

```
BE a, b, c
legnagyobb <- a
HA b > legnagyobb AKKOR
    legnagyobb <- b
HA c > legnagyobb AKKOR
    legnagyobb <- c
KI legnagyobb
```

(7, 3, 5) eredménye: 7
(4, 4, 4) eredménye: 4
(−2, −5, −3) eredménye: −2

A kezdőértéket a feladat szerkezetéből választjuk.

A hatodik hét kulcsfogalmai

Fogalom	A lényeg
ASCII és EBCDIC	Eltérő karakterkód-kiosztások
Unicode és UTF	Közös kódtér és annak reprezentációi
Gépi utasítás	Az ISA által meghatározott művelet
CISC és RISC	Utasításkészleti tervezési hagyományok
Algoritmus és program	Megoldási lépések és konkrét megvalósítás

Karakterkódolási háttér és dokumentáció

ASCII kódtábla és vezérlőkódok

https://www.rfc-editor.org/rfc/rfc20

EBCDIC IBM-037

https://www.ibm.com/docs/en/zos/3.1.0?topic=sequences-ebcdic

Unicode UTF kérdések

https://unicode.org/faq/utf_bom.html

C# char és string

https://learn.microsoft.com/en-us/dotnet/standard/base-types/character-encoding-introduction

Utasításkészletek és algoritmusfogalom

Intel x86 utasításkészlet

https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

RISC-V RV32I alaputasítások

https://docs.riscv.org/reference/isa/v20240411/unpriv/rv32.html

Algoritmus fogalma

https://xlinux.nist.gov/dads/HTML/algorithm.html

C# és a végrehajtási folyamat

https://learn.microsoft.com/en-us/dotnet/standard/managed-execution-process

Internetről származó szemléltető képek

Lyukkártya

Harke, CC BY-SA 3.0

https://commons.wikimedia.org/wiki/File:Punch_card_Fortran_Uni_Stuttgart_(3).jpg

IBM System/360

Erik Pitti, CC BY 2.0

https://commons.wikimedia.org/wiki/File:IBM_System360_Mainframe.jpg

Nyelvnevek saját írásrendszerükben

Runabhattacharjee, CC0

https://commons.wikimedia.org/wiki/File:Language_Autonym_Cloud.png

Az ASCII kódtáblája 1. rész

alsó hex	0x	1x	2x	3x	4x	5x	6x	7x
x0	NUL	DLE	SP	0	@	P	`	p
x1	SOH	DC1	!	1	A	Q	a	q
x2	STX	DC2	"	2	B	R	b	r
x3	ETX	DC3	#	3	C	S	c	s
x4	EOT	DC4	$	4	D	T	d	t
x5	ENQ	NAK	%	5	E	U	e	u
x6	ACK	SYN	&	6	F	V	f	v
x7	BEL	ETB	'	7	G	W	g	w

A felső hex számjegy az oszlopban, az alsó a sorban áll.

Az ASCII kódtáblája 2. rész

alsó hex	0x	1x	2x	3x	4x	5x	6x	7x
x8	BS	CAN	(	8	H	X	h	x
x9	HT	EM	)	9	I	Y	i	y
xA	LF	SUB	*	:	J	Z	j	z
xB	VT	ESC	+	;	K	[	k	{
xC	FF	FS	,	<	L	\	l	|
xD	CR	GS	-	=	M	]	m	}
xE	SO	RS	.	>	N	^	n	~
xF	SI	US	/	?	O	_	o	DEL

A felső hex számjegy az oszlopban, az alsó a sorban áll.

Az EBCDIC IBM-037 kiosztása 1. rész

hex	0x	1x	2x	3x	4x	5x	6x	7x	8x	9x	Ax	Bx	Cx	Dx	Ex	Fx
x0	CTL	CTL	CTL	CTL	SP	&	-	ø	Ø	°	µ	^	{	}	\	0
x1	CTL	CTL	CTL	CTL	NB	é	/	É	a	j	~	£	A	J	÷	1
x2	CTL	CTL	CTL	CTL	â	ê	Â	Ê	b	k	s	¥	B	K	S	2
x3	CTL	CTL	CTL	CTL	ä	ë	Ä	Ë	c	l	t	·	C	L	T	3
x4	CTL	CTL	CTL	CTL	à	è	À	È	d	m	u	©	D	M	U	4
x5	CTL	CTL	CTL	CTL	á	í	Á	Í	e	n	v	§	E	N	V	5
x6	CTL	CTL	CTL	CTL	ã	î	Ã	Î	f	o	w	¶	F	O	W	6
x7	CTL	CTL	CTL	CTL	å	ï	Å	Ï	g	p	x	¼	G	P	X	7

CTL: vezérlő SP: szóköz NB: nem törő szóköz SH: lágy kötőjel

Az EBCDIC IBM-037 kiosztása 2. rész

hex	0x	1x	2x	3x	4x	5x	6x	7x	8x	9x	Ax	Bx	Cx	Dx	Ex	Fx
x8	CTL	CTL	CTL	CTL	ç	ì	Ç	Ì	h	q	y	½	H	Q	Y	8
x9	CTL	CTL	CTL	CTL	ñ	ß	Ñ	`	i	r	z	¾	I	R	Z	9
xA	CTL	CTL	CTL	CTL	¢	!	¦	:	«	ª	¡	[	SH	¹	²	³
xB	CTL	CTL	CTL	CTL	.	$	,	#	»	º	¿	]	ô	û	Ô	Û
xC	CTL	CTL	CTL	CTL	<	*	%	@	ð	æ	Ð	¯	ö	ü	Ö	Ü
xD	CTL	CTL	CTL	CTL	(	)	_	'	ý	¸	Ý	¨	ò	ù	Ò	Ù
xE	CTL	CTL	CTL	CTL	+	;	>	=	þ	Æ	Þ	´	ó	ú	Ó	Ú
xF	CTL	CTL	CTL	CTL	|	¬	?	"	±	¤	®	×	õ	ÿ	Õ	CTL

CTL: vezérlő SP: szóköz NB: nem törő szóköz SH: lágy kötőjel